Phoenix

Inventory System

Complete Project Documentation & User Guide

React | Node.js / Express | REST API | JWT Auth | PostgreSQL

Table of Contents

What this deck covers

01 **Project Overview**
Purpose, goals & target users

02 **Key Features**
Core functionality at a glance

03 **System Architecture**
High-level design & data flow

04 **Tech Stack**
Tools, frameworks & libraries

05 **Project Structure**
Folder layout & codebase map

06 **Database Schema**
Tables, relations & data model

07 **API Reference**
Endpoints & REST conventions

08 **User Roles & Auth**
JWT, roles & permission matrix

09 **Getting Started**
Installation & setup guide

10 **User Guide**
How to use the application

11 **Future Roadmap**
Planned features & improvements

12 **Summary**
Quick recap & contacts

Project Overview

What is Phoenix Inventory?

About the Project

Phoenix Inventory is a full-stack web application designed to help retail businesses manage their product stock efficiently. It provides a centralized platform where teams can track inventory levels, manage product data, handle user access, and generate business reports — all from a modern, responsive interface.

Full-Stack

React + Node.js

Database

SQL (PostgreSQL/SQLite)

Auth

JWT-based Secure Login

Access

Multi-role User System

Key Features

Core functionality of the system

Product CRUD

Add, edit, delete products with full details including name, SKU, category, price & quantity.

Stock Alerts

Automatic low-stock notifications when inventory drops below defined threshold levels.

Search & Filter

Fast search by name, SKU, or category. Filter by stock status, price range, or supplier.

User Roles

Admin, Staff and Viewer roles with distinct permissions for secure, controlled access.

Reports & Export

Generate inventory summaries, movement history and export data to CSV for analysis.

System Architecture

3-tier client-server architecture

PRESENTATION TIER

React Frontend

- React.js (SPA)
- Axios HTTP Client
- React Router
- Context API / State

REST / HTTP

APPLICATION TIER

Node.js Backend

- Express.js Server
- JWT Middleware
- REST API Routes
- Business Logic Layer

REST / HTTP

DATA TIER

Database

- PostgreSQL / SQLite
- ORM (Sequelize/Knex)
- Schema Migrations
- Data Persistence

Browser → HTTP Requests → API Server → SQL Queries → Database

Technology Stack

Tools, frameworks & libraries used

Frontend

React.js
UI component library (SPA)

React Router
Client-side routing

Axios
HTTP request handling

CSS / Tailwind
Styling & responsive layout

Backend

Node.js
JavaScript runtime environment

Express.js
Web server & routing framework

JWT (jsonwebtoken)
Token-based authentication

bcrypt
Password hashing & security

Data & DevOps

PostgreSQL / SQLite
Relational database engine

Sequelize / Knex
ORM / Query builder

dotenv
Environment config management

Git / GitHub
Version control & collaboration

Project Structure

Codebase folder layout & organization

```
phoenix-inventory/
├── client/
│   ├── src/components/
│   ├── src/pages/
│   ├── src/context/
│   └── src/utils/
├── server/
│   ├── routes/
│   ├── controllers/
│   ├── models/
│   ├── middleware/
│   └── config/
└── .env / package.json
```

client/src/components/

Reusable UI components — Navbar, ProductCard, Modal, Table, Alert banners.

client/src/pages/

Top-level page views: Dashboard, Products, Login, Reports, Users.

server/routes/

Express route definitions — products, auth, users, reports endpoints.

server/controllers/

Business logic handlers for each route group.

server/models/

Database model definitions (ORM schemas & table mappings).

server/middleware/

JWT authentication guard, role-checking, error handler.

Database Schema

Tables, fields & relationships

users

PK	id
	name
	email
	password_hash
	role

products

PK	id
	name
	sku
	category
	price

stock_movements

PK	id
FK	product_id
FK	user_id
	type (in/out)
	quantity

categories

PK	id
	name
	description

API Reference

RESTful endpoints & conventions

Method	Endpoint	Description
POST	/api/auth/register	Register a new user account
POST	/api/auth/login	Authenticate & receive JWT token
GET	/api/products	List all products (with filters)
POST	/api/products	Create a new product
PUT	/api/products/:id	Update product details
DELETE	/api/products/:id	Delete a product record
GET	/api/products/low-stock	Get all low-stock products
GET	/api/reports/summary	Inventory summary report
GET	/api/reports/movements	Stock movement history
GET	/api/users	Admin: list all users
PUT	/api/users/:id/role	Admin: change user role

All protected routes require: Authorization: Bearer <token> header

User Roles & Authentication

JWT-based auth with role-based access control

JWT Authentication Flow

1. User submits email + password
2. Server validates credentials
3. bcrypt compares password hash
4. JWT token signed & returned
5. Client stores token (localStorage)
6. Token sent in every request header
7. Middleware verifies & decodes token
8. Role checked → Access granted/denied

Permission Matrix

Permission	Admin	Staff	Viewer
View Products	✓	✓	✓
Add / Edit Items	✓	✓	✗
Delete Items	✓	✗	✗
Manage Users	✓	✗	✗
View Reports	✓	✓	✓
Export Data	✓	✓	✗

Getting Started

Installation & local setup guide

01 **Clone the Repository**

```
git clone https://github.com/itphoenixpack/phoenix-inventory.git
cd phoenix-inventory
```

02 **Install Dependencies**

```
cd server && npm install
cd ../client && npm install
```

03 **Configure Environment**

```
cp .env.example .env
# Edit DB_URL, JWT_SECRET, PORT
```

04 **Run Database Migrations**

```
cd server && npm run migrate
npm run seed  # (optional demo data)
```

05 **Start the Application**

```
# Terminal 1 (backend)
cd server && npm run dev
# Terminal 2 (frontend)
cd client && npm start
```

Prerequisites: Node.js v16+ • npm v8+ • PostgreSQL v12+ (or SQLite for dev)

User Guide

How to use Phoenix Inventory day-to-day

Dashboard

- Log in with your credentials
- View KPI cards: total products, low-stock count, recent movements
- Click any card to drill into details

Managing Products

- Go to Products → click Add Product
- Fill in name, SKU, category, price, quantity & low-stock threshold
- Use Edit (pencil icon) or Delete (trash icon) per row

Search & Filter

- Use the search bar at the top of the Products page
- Apply filters: Category, Stock Status (In/Low/Out)
- Results update in real-time as you type

Reports & Export

- Navigate to Reports in the sidebar
- Choose Inventory Summary or Movement History
- Click Export CSV to download the data

Future Roadmap

Planned features & improvements

Phase 1

Near-term

- Email notifications for low stock
- Barcode scanner integration
- Bulk CSV import for products
- Mobile-responsive UI polish

Phase 2

Mid-term

- Supplier management module
- Purchase order tracking
- Multi-warehouse / location support
- Advanced analytics dashboard

Phase 3

Long-term

- REST API documentation (Swagger)
- Mobile app (React Native)
- AI-based demand forecasting
- Third-party ERP integrations

Roadmap priorities may shift based on user feedback and business needs.

Summary

- React + Node.js full-stack app
- RESTful API architecture
- JWT authentication & RBAC
- SQL database with ORM
- 5 core modules: Products, Roles, Reports, Search, Alerts

Repository

github.com/itphoenixpack/phoenix-inventory

Stack

React • Node.js • Express • PostgreSQL

Auth

JWT + bcrypt • Role-Based Access Control

API

RESTful • 11 core endpoints

Roles

Admin • Staff • Viewer

Thank You • Phoenix Inventory • itphoenixpack

For questions — raise a GitHub Issue or contact the dev team